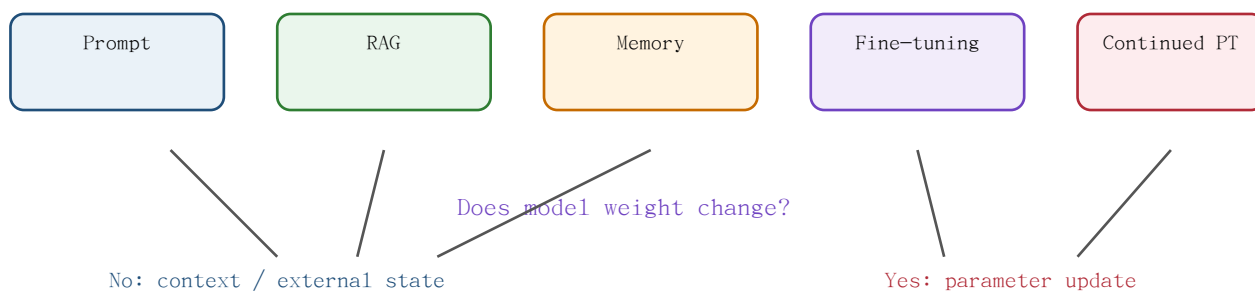


第33讲：Fine-tuning、LoRA 与 QLoRA

什么时候需要真正修改模型？——Prompt、RAG、Memory 与参数更新到底有什么区别

本讲目标

把 Prompt、RAG、Memory、SFT、Continued Pretraining、Full Fine-tuning、LoRA 和 QLoRA 放到同一张图里。学完后你应该能够判断：问题到底是“模型没看到信息”，还是“模型本身的长期行为需要改变”。



先问：问题是“缺上下文”还是“模型本身的行为/能力需要改变”？

先记住一句：如果只是缺知识或缺上下文，优先补

Context；如果希望模型跨请求长期改变输入到输出的映射，才考虑参数更新。

1. 五种“让模型表现不一样”的办法

方法	改权重?	主要作用
Prompt	否	本轮告诉模型怎么做
RAG	否	把外部知识送进当前 Context
Memory	否	跨轮次保存并检索经历
Fine-tuning	是	长期改变行为映射
Continued Pretraining	是	进一步适应领域文本分布

根本区别

Prompt/RAG/Memory 改的是“模型当前看见什么”；Fine-tuning 改的是“模型本身怎样响应输入”。

2. Prompt：成本最低，应该最先试

如果需求只是输出格式、回答风格、步骤约束、临时规则，Prompt 往往已经足够。

需求	Prompt 是否合适
输出 JSON	很合适
回答更简洁	很合适
本次任务遵循某规则	合适
永久掌握大量新知识	不合适
任何上下文下都稳定表现某行为	可能不够

最低成本原则

Prompt -> RAG/Tool/Memory -> 参数更新。不要因为 LoRA 流行，就把所有问题都变成训练问题。

3. RAG：知识在模型外面，按需取证

如果知识经常变化、属于私有文档、需要来源可追溯，RAG 往往比 Fine-tuning 更自然。知识更新时只需更新文档/索引，不必重新训练权重。

RAG 解决的是：模型这一轮需要看到什么外部证据，而不是让模型永久内化这些文档。

4. Memory：保存经历，不等于参数内化

Memory 适合保存用户偏好、Agent 的历史事件、成功/失败经验。未来检索回来以后，模型会表现得像“记住了”；但底层参数通常没有变化。

5. Fine-tuning：什么时候真的要动参数？

目标	为什么可能需要参数更新
稳定任务行为	希望跨请求反复出现
固定输出/工具策略	不想每次都靠长 Prompt

目标	为什么可能需要参数更新
领域任务映射	有稳定的监督输入-输出关系
小模型专项能力	希望把能力压进较小模型
稳定角色/策略差异	希望差异比 Persona Prompt 更持久

不要误用
如果问题是知识实时变化，Fine-tuning
往往不是首选，因为来源难追踪，而且每次更新都要重新训练。

6. SFT: 最常见的 Fine-tuning

SFT (Supervised Fine-Tuning) 给模型输入和理想输出，继续使用 next-token prediction / cross-entropy，只是训练数据变成了任务、指令或对话格式。

```
input -> desired output -> loss -> backprop -> update parameters
```

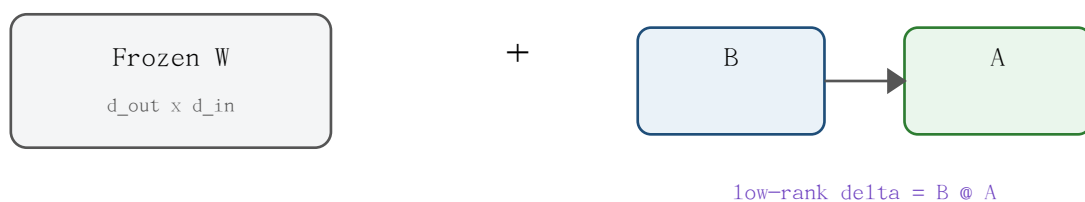
7. Full Fine-tuning: 为什么资源贵？

Full Fine-tuning 更新大量甚至全部模型参数。训练时除了权重，还要保存梯度、优化器状态和激活，所以显存远大于仅加载模型推理。

训练内存组成	Full FT
Weights	需要
Gradients	需要
Optimizer states	通常需要
Activations	需要

对资源有限研究者
通常不从大模型 Full Fine-tuning 起步，更常见的是 LoRA/QLoRA 或直接使用 API。

8. LoRA: 只训练低秩增量



$$W' = W + \alpha/r * (B @ A)$$

LoRA 不训练整个 W ，只训练低秩增量。

LoRA 把适配后的权重写成：

$$W' = W + \Delta W$$

再约束：

$$\Delta W = B @ A$$

其中 rank r 很小。原始 W 冻结，只训练 A 和 B 。它的核心假设是：很多任务适配所需的权重变化可以被低秩子空间近似。

9. LoRA 为什么省参数？

若 W 是 $d_{\text{out}} \times d_{\text{in}}$ ，完整矩阵参数数量为 $d_{\text{out}} * d_{\text{in}}$ ；LoRA 只训练 $r * d_{\text{in}} + d_{\text{out}} * r$ 。

例子：4096 × 4096	可训练参数
Full	16,777,216
LoRA rank=8	65,536
比例	约 0.39%

10. LoRA Rank：容量与成本的旋钮

rank 较小	rank 较大
训练参数少	训练参数增加
显存更低	适配容量更高
可能欠拟合复杂变化	更能逼近复杂 ΔW
适合快速实验	成本更高

Rank 不是越大越好；它和学习率、target modules 一样属于实验超参数。

11. LoRA 插在哪些层？

常见 target modules 包括 Attention 的 Q/K/V/O 投影，以及 FFN/MLP 的线性层。不同 target modules 会改变参数量、显存和最终效果，因此论文应明确报告。

12. QLoRA: Q 指的是什么？

QLoRA 的基本思路是：基础模型权重以低比特形式冻结加载，用更少显存保存 `base model`；其上仍训练 LoRA adapter。

quantized frozen base + trainable LoRA adapter

不要混淆
QLoRA 不是把整个 INT4 模型当普通 Full Fine-tuning 那样全部更新；核心是量化冻结 base + 参数高效适配。

13. QLoRA 为什么适合个人研究者？

优势	意义
低比特基础权重	显著减少模型加载显存
少量可训练参数	梯度/优化器状态规模更小
保留强 <code>base model</code>	不用从小模型重训
实验迭代较快	适合任务/领域适配

14. Adapter 保存了什么？

LoRA 训练后通常保存的是小体积 adapter 参数和配置，而不是重新保存一整份完整 `base model`。部署时可以加载 adapter，也可以在某些流程中 merge 回基础权重。

15. Continued Pretraining: 和 SFT 不一样

SFT	Continued Pretraining
指令/输入-输出数据	大量领域原始文本
强调任务行为	强调领域分布
告诉模型 “怎么回答”	更像让模型继续 “读这个领域的书”
监督结构较强	可以不需要任务标注

例子
若模型对某专业领域的术语和文本分布整体陌生，Continued Pretraining 可能有意义；若只是希望输出固定格式，SFT/Prompt 更直接。

16. Domain Adaptation 常常是组合路线

一个领域路线可能是：

Base Model -> Continued Pretraining -> SFT -> Preference / Verification

但不是每个项目都需要完整流水线。每多一阶段都意味着数据、训练和评测成本。

17. Catastrophic Forgetting: 新学会，也可能旧能力退化

参数更新可能让模型在新任务上更好，但原本的通用能力下降。缓解方式包括更小学习率、混合通用数据、PEFT、正则化和回归测试。

18. Overfitting: LoRA 也会过拟合

参数少不等于不会过拟合。小数据、高度重复的指令仍可能让模型只记模板。Train / Validation / Test 的基本纪律完全不变。

19. Fine-tuning 能不能“写入知识”？

参数训练确实可以让某些知识和模式更容易被生成，但它不是精准数据库写入。

实用区分
需要可更新、可追溯、精确事实 -> RAG; 需要稳定行为、任务映射、领域语言习惯 -> Fine-tuning 更自然。

20. 一张决策表：先选最小足够方案

问题	优先方案
只是改变格式/风格	Prompt
需要最新、私有、可引用知识	RAG
需要跨轮次保存经历	Memory
稳定任务行为反复出现	SFT / LoRA
领域语言分布差异很大	Continued Pretraining
资源有限但确实需要改权重	LoRA / QLoRA

21. Agent Systems 里什么时候 Fine-tune?

Agent 问题	可能方案
Tool schema 经常选错	SFT/LoRA on tool-use trajectories
希望 planner 形成稳定策略	SFT/LoRA
知识经常变化	优先 RAG
长期个体经历	优先 Memory
小模型需要专项角色	LoRA 可能合适
研究训练经历形成异质性	不同 adapter 可作为机制变量

22. Fine-tuning 与功能性异质性

Persona Prompt 产生的差异可能较浅且受上下文影响；不同训练数据或 adapter 可能形成更稳定的策略差异。

Training divergence -> behavior divergence -> error decorrelation -> collective gain

科研接口

LoRA 可以是制造 Agent 异质性的一种方法，但最终仍要证明差异是否稳定、是否由训练经历造成、是否降低错误相关性，以及相同计算预算下集体收益是否提高。

23. 小实验：低秩近似到底省了什么？

材料包实验构造一个 16×16 的权重更新 ΔW ，并让真实任务变化近似 $\text{rank}=2$ 。然后用 SVD 得到不同 rank 的最佳低秩近似。

方案	训练参数	关注点
Full ΔW	256	表达能力无约束
rank=1 LoRA	32	参数少但可能欠拟合
rank=2 LoRA	64	若真实变化近低秩，可很好逼近
rank=4/8	更多	误差进一步降低，成本也上升

24. 常见误区汇总

误区	更准确的理解
有新知识就要 Fine-tune	动态知识通常先考虑 RAG
LoRA = 不改模型	它仍训练参数，只是参数很少
QLoRA = 全模型 INT4 普通训练	量化冻结 base + LoRA
Rank 越大越好	需要容量/成本权衡
Fine-tuning 后一定更强	可能遗忘、过拟合或回归
Memory 写入 = 参数学习	外部存储与参数更新不同
不同 Role Prompt = 稳定异质性	必须实验证明长期行为和错误结构差异

25. 练习题

- 1) Prompt、RAG、Memory、Fine-tuning 的最根本区别是什么？
- 2) 什么场景下优先 RAG，而不是 Fine-tuning？
- 3) SFT 与 Continued Pretraining 有什么区别？
- 4) Full Fine-tuning 为什么显存成本高？
- 5) LoRA 的核心低秩思想是什么？
- 6) 为什么 LoRA 的可训练参数量远小于完整矩阵？
- 7) QLoRA 为什么更节省显存？
- 8) 什么叫 Catastrophic Forgetting？
- 9) 为什么 Fine-tuning 也必须严格划分 Train/Validation/Test？
- 10) 如果研究 Agent 功能性异质性，LoRA 可以怎样作为实验变量？

答案见材料包《03_第33讲_练习题与答案.pdf》。

26. 本讲总结

一句话总纲
Prompt/RAG/Memory 改的是“模型看到什么”；Fine-tuning 改的是“模型参数怎样响应输入”。LoRA/QLoRA 的价值，是用很少可训练参数完成这种长期行为适配。

方案	最短记法
Prompt	临时告诉模型怎么做
RAG	把外部知识拿进 Context
Memory	把经历留到未来检索
SFT	用理想输入-输出改变长期行为
Continued PT	继续适应领域语料
LoRA	训练低秩 ΔW
QLoRA	量化 base + LoRA

下一讲：第34讲《Open-source LLM：模型权重、本地部署和 API 到底有什么区别？》。